P1664R0
reconstructible_range - a concept for putting ranges back together

Published Proposal, 2019-08-05

Authors:

JeanHeyd Meneide

Hannes Hauswedell

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++20

Latest:

https://thephd.github.io/vendor/future_cxx/papers/d1664.html

Abstract

This paper proposes new concepts to the Standard Library for ranges called Reconstructible Ranges for the purpose of ensuring a range/view broken down into its two iterators can be "glued" back together using a constructor taking its iterator and sentinel type.

Table of Contents

§ 1. Revision History

§ 1.1. Revision 0 - August, 5th, 2019

— Initial release.

§ 2. Motivation

Currently

```
vector vec{1, 2, 3, 4, 5};
span s{vec.data(), 5};
span v = s | view::drop(1) | view::take(10)
           | view::drop(1) | view::take(10);
```

❌ - Compilation error

```
vector vec{1, 2, 3, 4, 5};
span s{vec.data(), 5};
auto v = s | view::drop(1) | view::take(10)
           | view::drop(1) | view::take(10);
```

⚠️ - Compiles, but `decltype(v)` is `ranges::take_view<ranges::drop_view<ranges::take_view<ranges::drop_view<span<int, dynamic_extent>>>>>`

```
std::u8string name = "AYLEMI·MVETEVIM ൗരൗ ·ലാശ ·ൾ";
char16_t conversion_buffer[432];

std::u8string_view name_view(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = std::text::transcode(input, output);
// compiler error here !!
std::u8string_view unprocessed_code_units
        = encoding_result.input;
std::span<char16_t> unconsumed_output
        = encoding_result.output;
```

❌ - Compilation error

```
std::u8string name = "AYLEMI·MVETEVIM ൗരൗ ·ലാശ ·ൾ";
char16_t conversion_buffer[432];

std::u8string_view name_view(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = std::text::transcode(input, output);
auto unprocessed_code_units = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

⚠️ - Compiles, but `decltype(unprocessed_code_units)` is `ranges::subrange<std::u8string_view::iterator, std::u8string_view::iterator>` and `decltype(unconsumed_output)` is `ranges::subrange<std::span<char16_t, std::dynamic_extent>::iterator, std::span<char16_t, std::dynamic_extent>::iterator>`

```
vector vec{1, 2, 3, 4,
span s{vec.data(), 5};
span v = s | view::dro
           | view::dro
```

✔️ - Compiles and works with r

```
vector vec{1, 2, 3, 4,
span s{vec.data(), 5};
auto v = s | view::dro
           | view::dro
```

✔️ - Compiles and works with r `std::span<int>`

```
std::u8string name = "
char16_t conversion_bu

std::u8string_view nam
std::span<char16_t> ou

auto encoding_result =
// compiler error here
std::u8string_view unp
        = encoding_res
std::span<char16_t> un
        = encoding_res
```

✔️ - Compiles and works, types

```
std::u8string name = "
char16_t conversion_bu

std::u8string_view nam
std::span<char16_t> ou

auto encoding_result =
auto unprocessed_code_
auto unconsumed_output
```

✔️ - Compiles and works, wher `std::u8string_view` and de `std::span<char16_t>`.

Currently in C++, there is no Generic ("with a capital G") way to take a range apart with its iterators and put it back together. That is, the following code is not guaranteed to work:

```cpp
template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
        using uRange = std::remove_cvref_t<Range>;
        if (std::ranges::empty(range)) {
                // ... the below errors
                return uRange(std::forward<Range>(range));
        }
        /* perform some work with the
        iterators or similar */
        auto first = std::ranges::begin(range);
        auto last = std::ranges::end(range);
        if (*first == u'\0xEF') {
                // ...
                std::advance(first, 3);
                // ...
        }
        // ... algorithm finished,
        // return the "updated" range!

        // ... but the below errors
        return uRange(std::move(first), std::move(last));
}

int main () {
        std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
        // this line will error
        std::string_view sub_view = operate_on_and_return_updated_range(meow_view);
        return 0;
}
```

The current fix is to employ `std::ranges::subrange<I, S, K>` to return a generic subrange:

```
template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
        using uRange = std::remove_cvref_t<Range>;
        using I = std::Iterator<uRange>;
        using S = std::Sentinel<uRange>;
        using Result = std::ranges::subrange<I, S>;
        if (std::ranges::empty(range)) {
                return uRange(std::forward<Range>(range));
        }
        // perform some work with the
        // iterators or similar
        auto first = std::ranges::begin(range);
        auto last = std::ranges::end(range);
        if (*first == u'\0xEF') {
                // ...
                std::advance(first, 3);
                // ...
        }
        // ... algorithm finished,
        // return the "updated" range!

        // now it works!
        return Result(std::move(first), std::move(last));
}

int main () {
        std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
        auto sub_view = operate_on_and_return_updated_range(meow_view);
        // decltype(sub_view) ==
        //   std::ranges::subrange<std::string_view::iterator,std::string_view::iterator>
        //   which is nowhere close to ideal.
        return 0;
}
```

This makes it work with any two pair of iterators, but quickly becomes undesirable from an interface point of view. If a user passes in a `std::span<T, Extent>` or a `std::basic_string_view<Char, Traits>` that interface and information is entirely lost to the user of the above function. `std::ranges::subrange<Iterator, Sentinel, Kind>` does not -- and cannot/should not -- mimic the interface of the view it was created from other than what information comes from its iterators: it is the barebones idea of a pair-of-iterators/iterator-sentinel style of range. This is useful in the generic sense that if a library developer must work with iterators, they can always rely on creation of a `std::ranges::subrange` of the iterator and sentinel.

Unfortunately, this decreases usability for end users. Users who have, for example a `std::string_view`, would prefer to have the same type after calling an algorithm. There is little reason why the original type needs to be discarded if it supports being put back together from its iterators. This break-it-apart-and-then-generic-subrange approach also discards any range-specific storage optimizations and layout considerations, leaving us with the most bland kind of range similar to the "pair of iterators" model. Compilation time goes up as well: users must spawn a fresh `std::ranges::subrange<I, S, K>` for every different set of iterator/sentinel/kind triplet, or handle deeply nested templates in templates as the input types. This makes it impossible to compile interfaces as dynamic libraries without having to explicitly materialize or manually cajole a `std::ranges::subrange` into something more palatable for the regular world.

There is also a problem where there are a wide variety of ranges that could conceivably meet this criterion, but do not. The author of the very first draft of this paper was not the only one to see utility in such operations. [p1739r0] does much the same that this paper does, without the introduction of a concept to formalize the behavior it presents. In particular, it selects views which can realistically have their return types changed to match the input range and operations being performed (or a similarly powerful alternative) by asking whether they are constructible from a subrange of the iterators with the expressions acted upon. This paper does not depend on any other papers, but note that the changes from [p1739r0], [p1391r2] and [p1394r2] all follow down to the logical conclusion laid out here:

— Ranges should be reconstructible from their iterators (or subrange of their iterators) where applicable;

— and, reconstructible ranges serve a useful purpose in generic algorithms, including not losing information and returning it in a much more cromulent and desirable form.

§ 3. Design

The design is simple and is given in 2 exposition-only concepts added to the standard:

```
template <typename R>
concept pair-reconstructible-range =
    Range<R> &&
    forwarding-range<std::remove_reference_t<R>> &&
    std::Constructible<R, iterator_t<R>, sentinel_t<R>>;

template <typename R>
concept reconstructible-range =
    Range<R> &&
    forwarding-range<std::remove_reference_t<R>> &&
    std::Constructible<R, std::ranges::subrange<iterator_t<R>, sentinel_t<R>>>;
```

It is the formalization that a range can be constructed from its begin iterator and end iterator/sentinel. It also provides an exposition-only concept for allowing a range to be constructed from a `subrange` of its iterator/sentinel pair. This allows a developer to propagate the input type's properties after modifying its iterators for some underlying work, algorithm or other effect. This concept is also the basis of the idea behind [p1739r0].

Both concepts require that the type with any references removed model the exposition-only concept `forwarding-range`. This ensures that the validity of the iterators is in fact independent of the lifetime of the range they originate from and that a "reconstructed" range does not depend on the original. We remove reference before performing this check, because all reference types that model `Range` also model `forwarding-range` and the intent of the proposed changes is narrower: (re)construction is assumed to be in constant time (this typically implies that `R` also models `View`, but it is sufficient to check `forwarding-range<std::remove_reference_t<R>>`). Note that this explicitly excludes types like `std::vector<int> const &` from being reconstructible.

§ 3.1. Should this apply to all Ranges?

Not all ranges can meet this requirement. Some ranges contain state which cannot be trivially propagated into the iterators, or state that cannot be reconstructed from the iterator/sentinel pair itself. However, most of the common ranges representing unbounded views, empty views, iterations viewing some section of non-owned storage, or similar can all be constructed from their iterator/iterator or iterator/sentinel pair.

For example `std::ranges::single_view` contains a exposition *semiregular-box* template type (ranges.semi.wrap) which holds a value to iterate over. It would not be possible to reconstruct the exact same range (e.g., iterators pointing to the exact same object) with the semi-regular wrapper.

§ 3.2. Applicability

There are many ranges to which this is applicable, but only a handful in the standard library need or satisfy this. If [p1391r2] and [p1394r2] are accepted, then the two most important view types -- `std::span<T, Extent>` and `std::basic_string_view<Char, Traits>` -- will model both concepts. `std::ranges::subrange<Iterator, Sentinel, Kind>` already fits this as well. By formalizing concepts in the standard, we can dependably and reliably assert that these properties continue to hold for these ranges. The ranges to which this would be helpfully applicable to in the current standard and proposals space are:

— `std::ranges::subrange` (already reconstructible)

— `std::span` (currently under consideration, [p1394r2]);

— `std::basic_string_view` (currently under consideration, [p1391r2]);

— `std::ranges::empty_view` (proposed here);

— and, `std::ranges::iota_view` (proposed here).

The following range adaptor closure objects will make use of the concepts in determine the type of the returned range:

— `view::drop` (currently under consideration, [p1035r6] and [p1739r0], re-proposed here);

— `view::take` (currently under consideration, [p1739r0], re-proposed here).

There are also upcoming ranges from [range-v3] and elsewhere that could model this concept:

— [p1255r4]'s `std::ranges::ref_maybe_view`;

— [p0009r9]'s `std::mdspan`;

— and, soon to be proposed by this author for the purposes of output range algorithms, [range-v3]'s `ranges::unbounded_view`.

And there are further range adaptor closure objects that could make use of this concept:

— `view::slice`, `view::take_exactly`, `view::drop_exactly` and `view::take_last` from [range-v3]

Note that these changes will greatly aid other algorithm writers who want to preserve the same input ranges. In the future, it may be beneficial to provide more than just an exposition-only concept to check, but rather a function in the standard in `std::ranges` of the form `template <typename Range> reconstruct(Iterator, Sentinel);`, whose goal is to check if it is possible to reconstruct the `Range` or otherwise return a `std::ranges::subrange`.

This paper does not propose this at this time because concepts -- and the things that rely on them -- must remain stable from now and into infinity. It is better as an exposition-only named set of helpers whose semantics we are a little more free to improve or be adapted, rather than hard-and-fast functions and named concepts which are impossible to fix or improve in the future due to the code which may rely on it.

§ 3.3. Two Concepts

By giving these ranges `Iterator, Sentinel`, and/or `std::ranges::subrange<Iterator, Sentinel>` constructors, we can enable a greater degree of interface fidelity without having to resort to `std::ranges::subrange` for all generic algorithms. There should be a preference for `Type(Iterator, Sentinel)` constructors, because one-argument constructors have extremely overloaded meanings in many containers and some views and may result in having to fight with other constructor calls in a complicated overload set. It also produces less compiler boilerplate to achieve the same result of reconstructing the range when one does not have to go through `std::ranges::subrange<I, S, K>`. However, it is important to attempt to move away from the iterator, sentinel model being deployed all the time: `std::ranges::subrange` offers a single type that can accurately represent the intent and can be fairly easy to constrain overload sets on (most of the time).

This paper includes two concepts that cover both reconstructible methods.

§ 4. Impact

Originally, the impact of this feature was perceived to be small and likely not necessary to work into C++20. Indeed: this paper originally targeted C++23 with the intent of slowly working through existing ranges and range implementations and putting the concept and the manifestation of concepts in range libraries, particularly range-v3, over time.

This changed in the face of [p1739r0]. Hauswedell's paper here makes it clear there are usability and API wins that are solved by this concept for APIs that are already in the working draft **today**, and that not having the concept has resulted in interface inconsistency and ad-hoc, one-off fixes to fit limited problem domains without any respite to routines which have a desire to preserve the input types into their algorithms. Since this paper's concept is likely to change interfaces API return values in a beneficial but ultimately breaking manner, this paper's consideration was brought up to be presented as a late C++20 paper for the purpose of fixing the interface as soon as possible.

Note that this is a separate concept. It is not to be added to the base `Range` concept, or added to any other concept. It is to be applied separately to the types which can reasonably support it for the benefit of algorithms and code which can enhance the quality of their implementation.

§ 5. Proposed Changes

The following wording is relative to the latest draft paper, [n4820], and to several papers whose utilities have not been completely placed into the C++ Working draft such as [p1739r0] and [p1035r6].

§ 5.1. Feature Test Macro

This paper results in an exposition-only concept to help guide the further development of standard ranges and simplify their usages in generic contexts. There is one proposed feature test macro, `__cpp_lib_range_constructors`, which is to be input into the standard and then explicitly updated every time a constructor from a pre-existing type is changed to reflect the new wording. We hope that by putting this in the standard early, most incoming ranges will be checked for compatibility with `pair-reconstructible-range` and `reconstructible-range`. This paper also notes that both [p1394r2] and [p1391r2] do not add wording for feature test macros, making this paper's feature test macro the ideal addition to encapsulate all reconstructible range changes.

§ 5.2. Intent

The intent of this wording is to provide greater generic coding guarantees and optimizations by allowing for a class of ranges and views that model the new exposition-only definitions of a reconstructible range:

— add a new feature test macro for reconstructible ranges to cover constructor changes;

— add two new exposition-only requirements to [range.req];

— add expression checks to `view::take` to reconstruct the range, similar to [p1739r0];

— add expression checks to `view::drop` to reconstruct the range, similar to [p1739r0] and from [p1035r6];

— add constructors for reconstructing the range to `view::empty_view`;

— add constructors for reconstructing the range to `view::iota_view`;

— and, make `view::iota_view` model *forwarding-range* by adding `friend` overloads of `begin()` and `end()`.

For ease of reading, the necessary portions of other proposal's wording is duplicated here, with the changes necessary for the application of reconstructible range concepts. Such sections are clearly marked.

§ 5.3. Proposed Library Wording

Append to §17.3.1 General [support.limits.general]'s **Table 35** one additional entry:

Macro name	Value	Header(s)
__cpp_lib_reconstructible_range	201907L	<string_view>, <ranges>,

Insert into §24.4.2 Ranges [range.range]'s after clause 7, one additional clause:

> § The exposition-only *pair-reconstructible-range* and *reconstructible-range* concepts denote ranges whose iterator and sentinel pair can be used to efficiently construct an object of the type from which they originated.

```
template <typename R>
concept pair-reconstructible-range =
    Range<R> &&
    forwarding-range<std::remove_reference_t<R>> &&
    std::Constructible<R, iterator_t<R>, sentinel_t<R>>;

template <typename R>
concept reconstructible-range =
    Range<R> &&
    forwarding-range<std::remove_reference_t<R>> &&
    std::Constructible<R, std::ranges::subrange<iterator_t<R>, sentinel_t<R>>>;
```

Add to §24.6.1.2 Class template `empty_view` [range.empty.view], a constructor in the synopsis and a new clause for constructor definitions:

```
namespace std::ranges {
  template<class T>
    requires is_object_v<T>
  class empty_view : public view_interface<empty_view<T>> {
  public:

    constexpr empty_view() noexcept = default;
    constexpr empty_view(T* first, T* last) noexcept;

    static constexpr T* begin() noexcept { return nullptr; }
    static constexpr T* end() noexcept { return nullptr; }
    static constexpr T* data() noexcept { return nullptr; }
    static constexpr ptrdiff_t size() noexcept { return 0; }
    static constexpr bool empty() noexcept { return true; }

    friend constexpr T* begin(empty_view) noexcept { return nullptr; }
    friend constexpr T* end(empty_view) noexcept { return nullptr; }
  };
}
```

24.6.1.3 Constructors **[range.empty.cons]**

```
empty_view(T* first, T* last) noexcept;
```

[1] Expects: `first == nullptr` and `last == nullptr`.

[2] Effects: none.

Add to §24.6.3 Class template `iota_view` [range.iota.view]'s clause 2, a constructor to the synopsis:

```
namespace std::ranges {
  template<class I>
    concept Decrementable =     // exposition only
      see below;
  template<class I>
    concept Advanceable =       // exposition only
      see below;

  template<WeaklyIncrementable W, Semiregular Bound = unreachable_sentinel_t>
    requires weakly-equality-comparable-with<W, Bound>
  class iota_view : public view_interface<iota_view<W, Bound>> {
  private:
    // [range.iota.iterator], class iota_view::iterator
    struct iterator;            // exposition only
    // [range.iota.sentinel], class iota_view::sentinel
    struct sentinel;            // exposition only
    W value_ = W();             // exposition only
    Bound bound_ = Bound();     // exposition only
  public:
    iota_view() = default;
    constexpr explicit iota_view(W value);
    constexpr iota_view(type_identity_t<W> value,
                        type_identity_t<Bound> bound);
```

```
    constexpr iota_view(iterator first, sentinel last);
```

```
    constexpr iterator begin() const;
    constexpr sentinel end() const;
    constexpr iterator end() const requires Same<W, Bound>;
```

```
    constexpr friend iterator begin(iota_view v);
    constexpr friend auto end(iota_view v);
```

```
    constexpr auto size() const
      requires (Same<W, Bound> && Advanceable<W>) ||
               (Integral<W> && Integral<Bound>) ||
               SizedSentinel<Bound, W>
    { return bound_ - value_; }
  };

  template<class W, class Bound>
    requires (!Integral<W> || !Integral<Bound> || is_signed_v<W> == is_signed_v<Bound>)
  iota_view(W, Bound) -> iota_view<W, Bound>;
}
```

Add to §24.6.3 Class template `iota_view` [range.iota.view], after clause 8, a constructor:

```
constexpr iota_view(iterator first, sentinel last);
```

[9] Effects: Equivalent to: `iota_view(*first, last.bound_)`

Add to §24.6.3 Class template `iota_view` [range.iota.view], after clause 11 (old) / 12 (new):

```
constexpr friend iterator begin(iota_view v);
```

[13] Effects: Equivalent to: `return v.begin();`

```
constexpr friend auto end(iota_view v);
```

[14] Effects: Equivalent to: `return v.end();`

§ 5.4. Proposed Library + P1739 wording

Modify §24.7.6.4 `view::take` [range.take.adaptor] as follows:

[1] The name `view::take` denotes a range adaptor object. ~~For some subexpressions E and F, the expression `view::take(E, F)` is expression-equivalent to `take_view{E, F}`.~~

[2] Let `E` and `F` be expressions, and let `T` be `remove_cvref_t<decltype((E))>`. Then the expression `view::take(E, F)` is expression-equivalent to:

- `T{ranges::begin(E), ranges::begin(E) + min<iter_difference_t<iterator_t<decltype((E))>>>(ranges::size(E), F)}` if `T` models `ranges::RandomAccessRange`, `ranges::SizedRange` and *pair-reconstructible-range*;
- `T{ranges::subrange{ranges::begin(E), ranges::begin(E) + min<iter_difference_t<iterator_t<decltype((E))>>>(ranges::size(E), F)}}` if `T` models `ranges::RandomAccessRange`, `ranges::SizedRange` and *reconstructible-range*;
- `ranges::take_view{E, F}` if that is well-formed;
- otherwise, `view::take(E, F)` is ill-formed.

§ 5.5. Proposed P1035 + P1739 wording changes

Modify [p1035r6]'s §23.7.8 `view::drop` as follows:

[1] The name `view::drop` denotes a range adaptor object. ~~For some subexpressions E and F, the expression `view::drop(E, F)` is expression-equivalent to `drop_view{E, F}`.~~

[2] Let `E` and `F` be expressions, and let `T` be `remove_cvref_t<decltype((E))>`. Then, the expression `view::drop(E, F)` is expression-equivalent to:

- `T{ranges::begin(E) + min<iter_difference_t<iterator_t<decltype((E))>>>(ranges::size(E), F), ranges::end(E)}` if `T` models ranges::RandomAccessRange, ranges::SizedRange and *pair-reconstructible-range*;
- `T{ranges::subrange{ranges::begin(E) + min<iter_difference_t<iterator_t<decltype((E))>>>(ranges::size(E), F), ranges::end(E)}}` if `T` models ranges::RandomAccessRange, ranges::SizedRange and *reconstructible-range*;
- `ranges::drop_view{E, F}` if the expression is well-formed;
- otherwise, `view::drop(E, F)` is ill-formed.

§ 6. Acknowledgements

Thanks to Corentin Jabot, Christopher DiBella, and Hannes Hauswedell for pointing me to p1035 and p1739 to review both papers and combine some of their ideas in here. Thanks to Eric Niebler for prompting me to think of the generic, scalable solution to this problem rather than working on one-off fixes for individuals views.

§ References

§ Informative References

[N4820]
Richard Smith. Working Draft, Standard for Programming Language C++. 18 June 2019. URL: https://wg21.link/n4820

[P0009R9]
H. Carter Edwards, Bryce Adelstein Lelbach, Daniel Sunderland, David Hollman, Christian Trott, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Mark Hoemmen. mdspan: A Non-Owning Multidimensional Array Reference. 20 January 2019. URL: https://wg21.link/p0009r9

[P1035R6]
Christopher Di Bella, Casey Carter, Corentin Jabot. Input Range Adaptors. 17 June 2019. URL: https://wg21.link/p1035r6

[P1255R4]
Steve Downey. A view of 0 or 1 elements: view::maybe. 17 June 2019. URL: https://wg21.link/p1255r4

[P1391R2]
Corentin Jabot. Range constructor for std::string_view. 17 June 2019. URL: https://wg21.link/p1391r2

[P1394R2]
Corentin Jabot, Casey Carter. Range constructor for std::span. 17 June 2019. URL: https://wg21.link/p1394r2

[P1739R0]
Hannes Hauswedell. Type erasure for forwarding ranges in combination with "subrange-y" view adaptors. 15 June 2019. URL: https://wg21.link/p1739r0

[RANGE-V3]
Eric Niebler; Casey Carter. range-v3. June 11th, 2019. URL: https://github.com/ericniebler/range-v3